Hurricane Alert and Resource Allocation for Managing Buffer-zone Evacuation (HARAMBE)

Hansaka Aluvihare, Jacob Edwards, Muhammad Najjar, Matthew Bivens, Yongxin Liu
Embry-Riddle Aeronautical University, FL 32114 USA,
aluvihah@my.erau.edu, edwarj47@erau.edu, najjarm@my.erau.edu, bivensm1@my.erau.edu, liuy11@erau.edu

***Abstract*—Artificial Intelligence (AI) is increasingly leveraged to address complex challenges across industries, including logistics during natural disasters. Florida's geography and infrastructure make it particularly vulnerable to transportation bottlenecks and supply chain disruptions, which hinder evacuation efforts and delay the delivery of critical supplies. This paper investigates the application of Deep Q Networks (DQN) to optimize flight diversion routes in response to hurricane impacts, with the goal of improving evacuation efficiency and reducing diversion costs.**

Building on previous work that employed optimization algorithms like PySwarms [1], we utilize DQN to enable the model to learn from simulated scenarios guided by a reward function designed to identify optimal flight diversion strategies. Our findings show that DQN consistently outperforms a baseline RNN model, which, although faster in initial training, was slower and less consistent in convergence. DQN's selective attention mechanism enhances the model's interpretability, aligning with the shift toward explainable AI by providing deeper insights into the decision-making process, a critical advancement for unpacking traditionally black-box models.

I. Introduction

Machine Learning (ML) has become a very proficient way of making predictions using our computational resources to best overcome and adapt to real-life scenarios. Many of said scenarios can be cumbersome due to the copious quantities of data that must be sifted through to create a valuable and well-trained model. Our project focuses on leveraging ML to analyze and predict patterns within airport traffic data, specifically using datasets provided by the FAA's Traffic Flow Management System Counts (TFMSC) website. This data offers valuable insights into airport operations, including flight volumes, passenger capacity per aircraft, and other metrics that impact the aviation industry. However, the dataset originally included numerous columns that were extraneous to our analysis. Sifting through these columns to extract only the most relevant features required careful consideration and domain knowledge, ensuring that the dataset was streamlined for modeling without losing critical information. The HARAMBE model or Hurricane Alert and Resource Allocation for Managing Buffer-zone Evacuation, is a model that will automatically choose what resource, in other words available aircraft/ flight, can be allocated to the zone that is going to be impacted by the approaching hurricane. For training and testing as of now, our project is using the data acquired during Hurricane Ian in 2022. Analyzing our model further, the HARAMBE model represents a novel application of ML in disaster management and resource optimization. By analyzing historical data and leveraging the predictive capabilities of ML, this model will be able to produce the results necessary to meet our goal. During Hurricane Ian in 2022, significant challenges arose in managing evacuation efforts due to the unpredictable nature of the storm and the high demand for flights out of affected areas. The data collected from this event provided a unique opportunity to train our model to better understand the patterns and dynamics of such scenarios, enabling us to develop a system that could mitigate chaos and improve coordination in future emergencies. The overarching objective of the HARAMBE model is to create a tool that not only saves lives but also optimizes the use of resources in a cost-effective manner. In the context of Florida, a state particularly vulnerable to hurricanes, this model could significantly enhance the ability of airports and airlines to respond to natural disasters. By predicting demand, identifying available resources, and recommending efficient allocation strategies, the model could help mitigate injuries & death, minimize financial strain on airlines & passengers, and ensure a more organized evacuation process. In our research, there was a point that was made regarding the importance of an alert system in times of necessity, and there aren't many times that the masses are in need in the U.S. other than during major natural disasters. Specifically, one of our research articles said, "A trauma resource allocation model can improve the geographic distribution of emergency services, enhancing readiness during disasters" [2] . With our model, governments could mitigate widespread panic during the times natural disasters strike. For example, our model is focused on the Hurricane Ian path that hit the west coast of Florida and moved northeast through the state and into the Atlantic Ocean. This historical data is something that can be used in combination with other historical instances of natural disasters to encompass a better understanding of how these storms move and analyzing them will allow us to predict and effectively use our resources to best assist those who will be directly affected. As a general practice before implementation, "Simulation models for resource allocation in multi-project scenarios offer valuable insights for disaster response planning, enabling efficient use of available resources" [3]. The model architecture that has been built in the HARAMBE project is one that allows us to simulate different scenarios with different hurricane weather & flight data as we continue to adapt to the different catastrophes and make our model more robust. If we were to have this model moved into a production state after receiving funding to continue our project, we would be able

to reduce a multitude of different costs other than just the current structure of the model, which is currently focused on saving fuel costs. "Cost-aware optimization of flight resources in disaster scenarios ensures timely evacuation while managing economic constraints" [4]. If we were unable to provide a cost-efficient methodology of implementing our model, then it wouldn't feasibly work in our capitalistic society; however, our model does provide the most advantageous course of action for both the airports and the airlines associated with it.

II. Related Work

Evaluation of the aircraft fuel economy using advanced statistics and machine learning, particularly concerning fuel efficiency, and the points below explores how a Recurrent Neural Network (RNN) model could be used to potentially produce better results and generate new outcomes compared to other Machine Learning (ML) models. The article emphasizes the importance of fuel efficiency in the airline industry and the challenges in accurately assessing the impact of retrofits on fuel consumption. Let's examine the key aspects and how an RNN could be beneficial:

- Fuel Economy Monitoring: The article introduces three key performance indicators commonly used to monitor fuel efficiency: Specific Range (SR), Fuel Used (FU), and Fuel Burn Off (FBO). These indicators rely on data collected during flight, which can be influenced by various factors such as measurement errors and turbulence. [5]
- RNN Application: An RNN could be trained on historical flight data, including fuel consumption, flight parameters (speed, altitude, etc.), and weather conditions. The RNN's ability to process sequential data makes it well-suited to analyze time-series data like flight data, potentially learning complex relationships between these factors and fuel efficiency. This could lead to more accurate predictions of fuel consumption for specific flight routes and conditions.
- Measurement Errors and Uncertainties: The article discusses different types of measurement errors, including systematic, dynamic, and stochastic errors, that can affect fuel efficiency calculations. Turbulence is also highlighted as a factor that introduces uncertainties. [5]
- RNN Application: RNNs can be effective in dealing with noisy data. They can learn to filter out noise and identify underlying patterns in the data, potentially mitigating the impact of measurement errors on fuel efficiency predictions. Furthermore, by incorporating turbulence data as an input feature, the RNN could potentially learn to account for its impact on fuel consumption. [5]
- Quantification of Fuel Efficiency Increase of Retrofits: The article focuses on the difficulties in accurately quantifying the fuel efficiency gains from implementing retrofits. This is particularly challenging for retrofits with small performance improvements, which can be masked by measurement errors and other uncertainties.
- RNN Application: An RNN could be valuable in this area. By training an RNN on data from flights with and without specific retrofits, the model could learn to isolate the effect of the retrofit on fuel consumption, even if the improvements are small. This could lead to more confident assessments of the effectiveness of different retrofits.
- Data-Based Evaluation Methods: The article discusses using machine learning models, like random forests, to simulate fuel flow and evaluate fuel economy. It highlights the importance of data quality for these models.
- RNN Application: While the article focuses on random forests, an RNN could be a powerful alternative for modeling fuel flow. RNNs excel at capturing temporal dependencies, which are crucial for understanding how fuel consumption evolves throughout a flight. An RNN could potentially outperform other ML models in predicting fuel flow, particularly in scenarios with fluctuating flight conditions.
- New Outcomes and Insights: An RNN's capacity to learn complex patterns could potentially reveal new insights into the factors influencing fuel efficiency. This might involve identifying previously unknown correlations between flight parameters, weather conditions, and fuel consumption, or uncovering subtle ways in which retrofits impact fuel use under different operational conditions. [5]

If our model were to have more features available such as in-flight changes of weather, engine performance, and exact route mileage, then we could produce an even more robust outcome for these resources to be chosen with a finer tooth comb, if you will.

The article, Improved algorithms for the evacuation route planning problem by Mishra, Mazumdar & Pal, provides context to a similar problem that they faced in their model evaluation and problem analysis. The article describes how increasing turbulence intensity leads to a wider spread of fuel flow values. This increased variability makes it difficult to distinguish between the inherent fluctuations due to turbulence and the potential fuel savings resulting from a retrofit. [6]. When we were evaluating the different costs associated with our idea, we found that one of the easiest problems to address could be fuel. However, we were quickly shown that it's an approximate cost due to the lack of data needed to truly encompass a perfect prediction of the fuel cost due to factors that are unaccounted for with the flight path such as turbulence, fluctuations in engine performance, age of the aircraft, etc. Our model is primarily using an approximated nautical miles per gallon ratio along with the geolocated distances between each of the airports vs. Flight paths produced during that time frame that provide a real mileage. These approximations are good for producing the predictions at this stage in our research; however, if we were to continue our research, then we should look to increase our parameters that we are considering for optimal fuel efficiency. Real-world constraints are able to be considered allowing the model to be more robust in its recommendations.

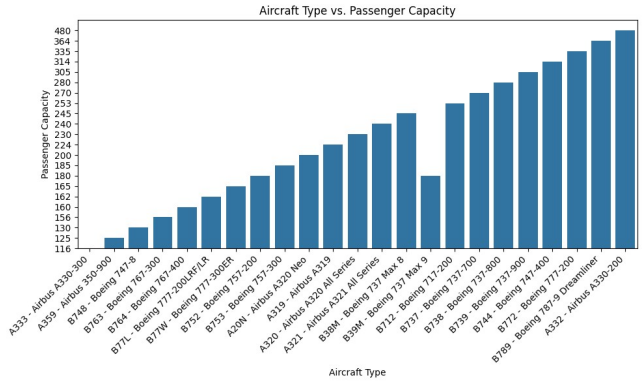

Fig. 1. Aircraft Type vs Passenger Capacity

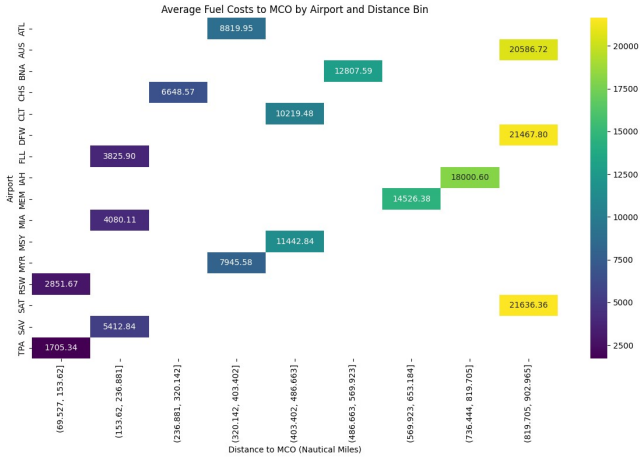

Fig. 2. Average Fual Costs to MCO by Airport and Distance.

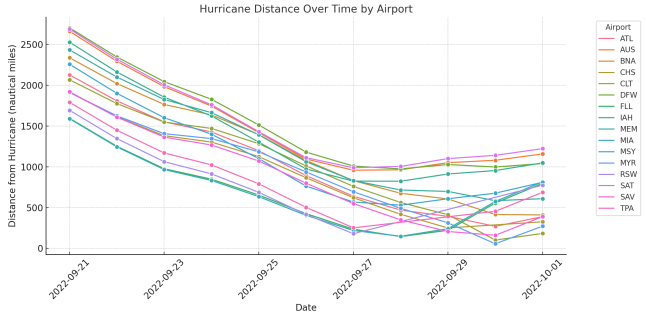

Fig. 3. Hurricane distance versus date by airport. As the hurricane gets closer to the airports, flight operations will be impacted.

III. Methodology

This paper delves into the complex domains of Deep Q-Learning and Deep Q Networks (DQN), exploring their theoretical foundations and practical implications for the advancement of artificial intelligence. Q-learning collects states, actions, and rewards from observations to train the algorithm to make future decisions. This method attempts to approximate an action-value function while remaining stable and reducing reliance on specific values.

When Q-learning is combined with neural networks, the result is a Deep Q network. Rather than storing value references in a Q-table, this method trains and predicts with a model. This model serves as a regression tool, producing output values for every possible action. These output values are usually continuous float numbers that represent the desired Q values. The following section describes the data preprocessing steps needed to create a dataset for algorithm training.

Proper data pre-processing involves filtering out unnecessary features, handling missing or erroneous data, and transforming the dataset into a form that aligns with the goals of the analysis. This step is critical, as the quality of the input data significantly impacts the accuracy and reliability of the resulting model. In the context of our project, this preprocessing stage was a vital component of preparing the dataset for effective training and testing. The dataset obtained from the FAA's TFMSC database provided us with a lot of information; however, it had some columns that were irrelevant to our model. It needed to be cleaned up by removing zero values from the departures as those were not necessary since we are focused on expanding the departures associated with each aircraft type at each airport during the time frame of September 21st, 2022, through October 1st, 2022. Once each of these departures had been expanded to create its own flight record of a departure, then we could start identifying the geographic distances, measured in Nautical Miles, between each of the 16 different airports. The airport's nautical mileage is calculated using the geopy library in Python. It corresponds the distance from MCO (Orlando International Airport) to each of the respective airports in dataset. Once this data was calculated, We used the cost of Jet-A Fuel during that time frame, which was when Hurricane Ian hit Florida. Using these two columns, the fuel cost associated with each flight was able to be calculated as an important field for our model. The model will use the costs associated with the fuel to predict and choose what flight option, or resource in this context, is best suited for extracting the maximum number of people out of the impending impact zone whilst keeping fuel costs at a minimum. 1 is a visualization of Aircraft Type vs Passenger Capacity. We can infer that there is a clear relationship between the size/type of aircraft and its ability to carry passengers, with wide-body planes accommodating more passengers than narrow-body planes. We observe from 2 that airports with shorter distances to MCO (e.g., within 153.62 nautical miles) have significantly lower fuel costs (e.g., $1,705 for TPA). As the distance to MCO increases, the fuel costs rise substantially, with costs exceeding $20,000 for distances greater than 819.70 nautical miles. Airports with longer distances to MCO (e.g., ATL, MEM) are clustered in higher distance bins, and these naturally incur higher fuel costs due to the longer flight paths. The dataset regarding the hurricane was sourced from NHCs archive of datasets. It consists of model predictions of the path and intensity of hurricane Ian from several models. The predictions are made every 6 hours of a day from September 20th 2022 through

October 1st 2022. We first condensed the dataset from all the future predictions to 24 hours from midnight of each day beginning from the 21st of September. Further feature selection and creation included removing unwanted features as well as creating a metric of 'Coverage Area', which gave us a rough distance of impact from the center. We then averaged latitude and longitude to give us a minimum and maximum prediction for the center of the hurricane on the day. This gave us the base to calculate the distance of each airport from the hurricane and it's intensity by coverage. 3 Shows us airports farther away from the hurricane's initial path (e.g., ATL, MEM) maintain relatively larger distances throughout the timeline. Airports like MYR and SAV, located closer to the hurricane's path, show shorter distances throughout. By the end of the timeline (September 29–30), the distance variation among airports reduces, indicating that the hurricane impacted multiple regions similarly. Certain airports have a noticeably slower rate of decreasing distance, suggesting they were farther from the main path of the hurricane.

A. Deep Reinforcement Learning for Flight Selection

The DQN algorithm approximates a previously learned Q-function. This forecasts future behavior in discrete spaces. The system takes initial states as input and uses actions to generate all possible Q-value outcomes. In our context, the input size is constantly changing, and the action space follows suit. The output layer will correspond to the agent's potential actions. This agent choses the action with the highest Q-value. The Bellman Equation, denoted as Equation 1, is used to generate new Q-values. We have employed this Bellman equation to calculate the actual Q-value based on the immediate reward received, denoted as r.

$$Q(s,a) = (1-\epsilon)Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right] \tag{1}$$

The exploration-exploitation trade-off parameter is ϵ, and the current estimate of predicted rewards is $Q(s,a)$. The learning rate is α, the immediate reward is r, the discount factor is γ, and the next state and action are s' and a'. The equation 1 includes two components: immediate reward (r) and maximum expected future rewards for the next state ($max(Q(s',a'))$. The variables (a and (s) denote actions and states, respectively. The Q-network is used to calculate the maximum future reward for each potential action. Following the initialization of a Q-table and execution of an algorithm, the agent performs a random action with constant updates. Over time, an optimal path is identified.

We used the following notation to formulate the reinforcement-learning problem. The agent is an entity that interacts with its environment and makes decisions in order to achieve a goal. It learns to perform tasks based on feedback from rewards. The agent can take certain actions regardless of its current state. The action space is a scalar representation of the environment's dynamics at each time step t. Thus, action can be defined as follows.

$$a(t) = a \in \mathbb{R}^{k_t}$$

Where k_t is the number of available flights at time step t.

The agent may receive a reward for their actions. If the agent selects the best flight available and receives a significant positive reward. In addition, if the agent took the incorrect flight, the reward will be reduced. In a Markov environment, the reward function guides the agent's discovery of the best policy. The agent's ultimate goal is to maximize the overall reward at the end of each episode. The reward function is shown in equation 2.

In this study, the environment is modeled using the Markov Decision Process (MDP). MDPs formalize the environment for reinforcement learning. In this fully observable environment, the agent has complete access to and control over the system's state. The MDP can be formulated as shown below.

$< S, A, P, R, \gamma >$

Where,

S – Finite set of states

A – Finite set of actions

P – State transition Probability Matrix

R – Reward Function

γ – Discount Factor

t – Time

In our case, we assume P equals 1 because we can observe every state of the agent. The reward function for action a in state s can be defined as follows.

$$R_s^a = E[R_{(t+1)}|S_t = s, A_t = a] \tag{2}$$

And we provide the immediate reward $R(t+1)$ based on the following function.

$$R(t+1) = 100p_f - \frac{d_f}{1000} - \frac{c_f}{100} + \frac{hd_f}{100} \tag{3}$$

Where p_f is the number of passengers on the selected flight f at time step t, d_f is the distance to the airport, c_f is the total cost of the flight f, and hd_f is the hurricane distance from the center of the hurricane eye. With its reward function, the agent will act to maximize the reward, resulting in an increase in passenger count and distance to the hurricane while reducing travel distance and total cost during resource allocation.

In the next time step $t+1$, the state vector (S_{t+1}) will be fed into the deep neural network to estimate the decision. The DQN uses the ϵ'-greedy policy to balance exploration and exploitation. This means that the agent follows the ϵ-greedy policy. The experience replay memory retains all previous experiences, including transitions (S_t, a_t, r_t, S_{t+1}). To break the correlation, episodes are divided into transitions and shuffled randomly. Following the encoder's forward propagation, a transformer neural network is used to estimate the action values $Q(S_t, a_t)$. The optimal or maximum $Q^*(S_t, a_t)$ value determines the agent's next action.

The encoder-only transformer is used as a function approximator to estimate the action values at each time step. To update network parameters, define the loss function as the mean squared error (MSE) between predicted and target Q values. Using a single encoder-only transformer to predict and target Q values can result in weight blow-ups. To estimate action values, we use two different neural networks: a frozen network and an updating network. Using the freeze encoder-only transformer, we can calculate $Q_{target}(S, a)$. The weights of the updating network could be updated at each iteration using the loss function below.

$$L(\omega) = E_{S,a,r,S' \sim D}[(r + \gamma.Q(S', a', \omega^{-}) - Q(S, A, \omega))^2]$$

D represents the experience replay memory, and r is the immediate reward from the reward function 3. ω represents the network parameters that are being updated, while ω^{-} represents the network parameters for freezing. Using gradient descent, we can iteratively improve the above MSE between predicted Q values and actual Q learning targets. After a few iterations, frozen network parameters can be copied and changed.

B. Encoder-only Transformer

We used an encoder-only Transformer architecture to estimate the Q-value function for the dynamic flight allocation problem. The encoder-only Transformer is ideal for scenarios with variable input sizes, such as a changing number of available flights at each time step. Unlike fully connected neural networks, which have a fixed input size, the Transformer can handle variable-length sequences using positional encodings and self-attention mechanisms, making it ideal for this problem.

The state at each time step is represented as a sequence of flight feature vectors, where each vector encapsulates 8 attributes: *Nautical MPG, Average Fuel Cost, Passenger Capacity, Distance MCO(Orlando) Airport, Estimated Fuel Cost to MCO(Orlando), Coverage Area, and Distance from hurricane* for a specific flight. To ensure compatibility with the Transformer, the state is padded to a fixed maximum sequence length N per batch, with the padded entries representing invalid flights. A binary mask M is used to distinguish valid flights from padded entries, ensuring that the self-attention mechanism focuses only on valid entries. The transformer encoder uses the input sequence $X \in \mathbb{R}^{N \times d_{\text{features}}}$, where d_{features} denotes the feature dimension of each flight. The sequence is embedded using a linear transformation$E = XW + b$. This enables the network to compute relationships between flights and identify the most important features for Q-value estimation. By stacking multiple attention layers and feedforward networks, the Transformer encoder learns hierarchical representations of the input state. The transformer encoder's output, $H \in \mathbb{R}^{N \times d_{\text{model}}}$, contains contextualized embeddings for each flight. To predict Q-values, a final linear layer converts these embeddings to scalar Q-values for each flight:

$$Q(s, a_i) = H_i W_{\text{out}} + b_{\text{out}}, \quad i = 1, \ldots, N,$$

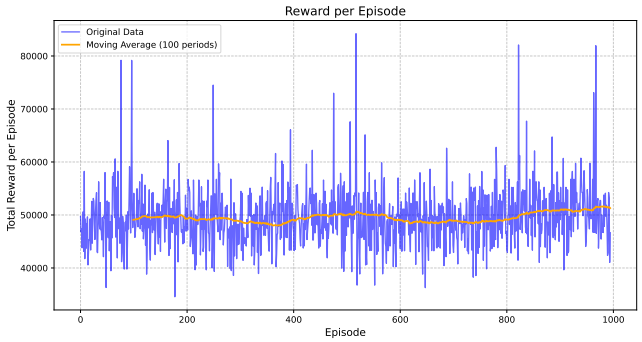

Fig. 4. Visualization of the average reward per episode of the transformer model, highlighting the model's performance and learning progression over time.

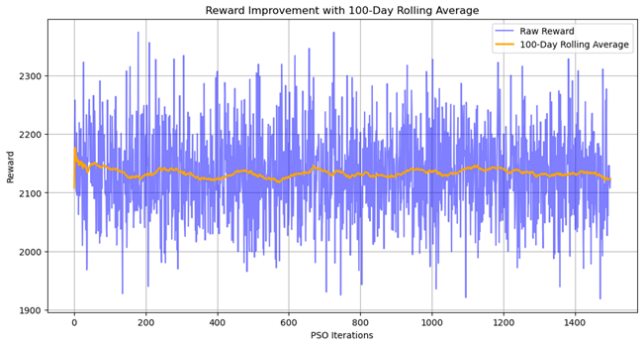

Fig. 5. Visualization of the average reward per episode of the RNN model, highlighting the model's performance and learning progression over time.

$Q(s, a_i)$ represents the predicted Q-value for the i-th flight. This layer's output dimension matches the number of valid flights, which is dynamically determined by the mask (M). The encoder-only Transformer architecture has several advantages for the flight allocation problem. First, its ability to handle varying input sizes allows it to adapt to the changing nature of available flights. Second, the self-attention mechanism detects relationships between flights, allowing for more informed decisions. Finally, masking improves computational efficiency while preventing interference from padded entries. The proposed architecture strikes a good balance between flexibility, scalability, and performance, making it an excellent choice for Q-value prediction in reinforcement learning tasks involving dynamic state and action spaces.

IV. Simulation Results & Discussion

Extensive simulations were carried out under various scenarios to assess the performance of the proposed transformer-based Q-value prediction network for the dynamic flight assignment problem. The simulations were intended to mimic real-world conditions, with varying numbers of available flights at each decision-making stage. The objective was to assess the model's ability to reduce operational costs while increasing passenger capacity by selecting the most efficient flights. The results were compared to baseline methods, such

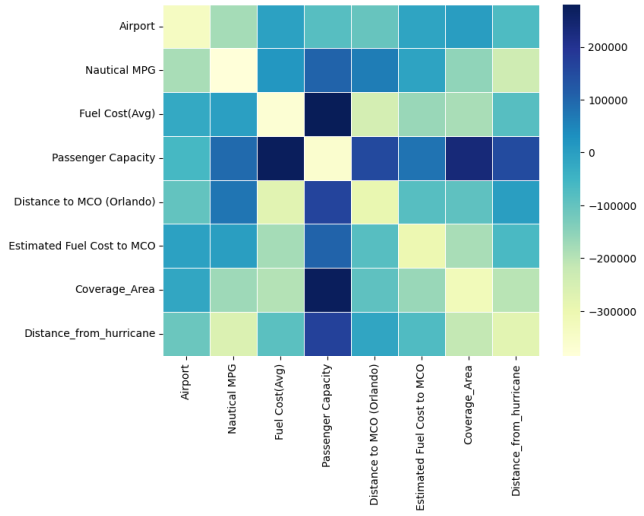

Fig. 6. Visualization that represents the scores that will be assigned when multiple inputs are given to the attention matrix. Higher values indicate it will give high importance to those features. Low means it will give low importance.

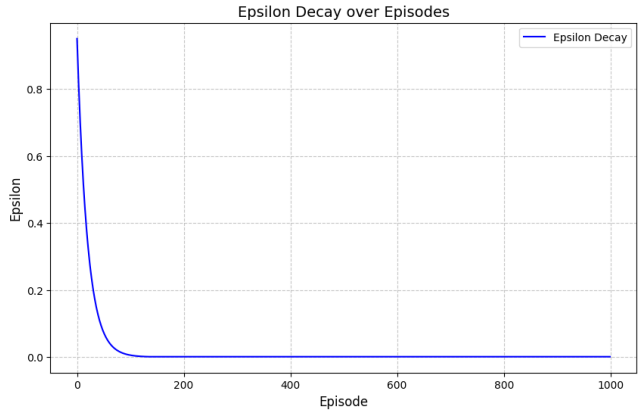

Fig. 7. This illustrates the epsilon curve, which shows the amounts of exploration and exploitation.

as a conventional recurrent neural network, to demonstrate the benefits of our proposed method.

The model is built with PyTorch and consists of two encoder layers with two parallel heads. We use a 64-dimensional embedding, with an 8-dimensional feature. Following the two encoder blocks, we use a linear layer to map the output to the appropriate Q value for each flight. As a result, the output dimension is one, with a linear activation function. To optimize, we use the Adam optimizer with a mini-batch size of 16 and a starting learning rate of 0.001.

The use of Experience Replay adds a significant level of sophistication. The replay memory of size 10000 functions as a memory buffer, storing previous transitions. This strategy reduces temporal correlation between consecutive samples during training, resulting in increased data efficiency and robust learning. To summarize, Table I contains a list of all hyperparameters used in the algorithm.

The experiments used a dataset described in Section III to

TABLE I
HYPERPARAMETERS OF THE DQN MODEL

Hyperparameter	Value
Batch Size	16
Gamma	0.99
Learning Rate	0.001
Initial Epsilon	1.0
Minimum Epsilon	0.001
Epsilon Decay	0.95
Replay Memory Size	10000
Num or Episodes	1000

observe each flight's features. We used available data from three airports (ATL, DFW, and CLT) to identify available flights. Each day, the number of available flights fluctuated dramatically. For the sake of simplicity and because the procedure for selecting the best flight is the same, we chose three consistent best flights from the available options. In our simulation, a single step represents one day, and the agent makes decisions based on the available flights on that particular day.

The reward function was carefully designed to strike the right balance between cost minimization and capacity maximization. Specifically, a positive reward was given for selecting flights with high passenger capacities and low costs, while penalties were imposed for exceeding capacity or selecting unsuitable flights.

The proposed Transformer-based Q-network outperformed the baseline RNN. After 800 epochs, the algorithm outperformed the RNN model in terms of total reward per episode. Furthermore, it consistently outperformed the RNN model, which frequently failed to adapt to the problem's dynamic nature. These improvements can be attributed to the Transformer's self-attention mechanism, which effectively captured relationships between flights and prioritized those that contributed the most to the reward function. The Transformer model's ability to handle variable-length inputs without requiring fixed input sizes increased its suitability for this problem.

The agent's learning curve, shown in Fig. 4, measured in terms of cumulative reward over training episodes, showed an increase in the average moving average after 800 epochs. The epsilon values during 1000 epochs are shown in 7. We allow the agent to take random actions in the early epochs, gradually decreasing the epsilon value. This approach encourages the agent to learn to explore instead of simply exploiting. This suggests that the model successfully learned to optimize its policy over time. We also conducted an experiment with a RNN that utilizes an input layer of 6 features (MPG, fuel cost, passenger capacity, distance, estimated fuel cost, hurricane proximity). The RNN, with 64 hidden units, processes flight data sequentially to a fully connected output layer to predict Q-values. A Particle Swarm Optimizer (PSO) is implemented to optimize the RNN weights, maximizing the reward function. This architecture combines sequence modeling and PSO to generate optimal flight recommendations. In comparison, the

RNN-based learning curve in 5 showed slower convergence and lower final cumulative rewards, primarily due to its inability to capture dependencies between flights efficiently.

One significant advantage of the proposed model is its interpretability using attention mechanisms. By analyzing the attention weights of the first layer for a randomly generated data sample as shown in Fig. 6, we were able to identify the features and flights that contributed most to the Q-value prediction. For example, flights with larger passenger capacities consistently received more attention, as expected. In addition, the total cost and hurricane distance of the flights will receive the most attention during forward propagation. This reflects the cost, and hurricane distance has a direct impact on the agent's actions. This is consistent with the reward structure and validates the model's decision-making process. Visualization of attention weights during specific decision steps revealed that the model's focus was dynamically adjusted based on the state, demonstrating adaptability and robustness.

In terms of computational efficiency, the Transformer-based Q-network is slower, taking only 170 minutes to train 1000 episodes, whereas the RNN model took only 70 minutes. Thus, training a transformer network is approximately 58% slower than the RNN network. However, The transformer's scalability was also beneficial, as it handled larger input sizes with little degradation in performance.

In contrast, the simulation results demonstrate the effectiveness of the proposed transformer-based Q-network in solving the dynamic flight allocation problem. Its ability to handle varying input sizes, capture complex relationships between flights, and provide interpretable decision-making makes it an effective tool for this application. Transformer models' superior performance in terms of cost reduction, capacity maximization, and adaptability demonstrates their potential for reinforcement learning tasks with dynamic state and action spaces. These findings pave the way for further investigation and application of transformer-based models in real-world logistics and resource allocation scenarios.

V. Conclusion

This study highlights the potential of leveraging advanced machine learning techniques, particularly transformer-based Deep Q Networks (DQN), to optimize flight resource allocation in disaster scenarios such as hurricanes. The proposed model demonstrated its ability to handle dynamic input sizes, capture complex relationships between flights, and provide interpretable decision-making. It outperformed baseline models, such as RNNs, in key metrics like cost reduction and passenger capacity maximization, offering a significant advantage in disaster response logistics. The simulations showed that the DQN model could effectively balance operational costs with evacuation efficiency, making it a valuable tool for improving disaster preparedness and response. While the model exhibited slightly slower training times compared to RNNs, its scalability and superior performance underscore its suitability for real-world applications. Furthermore, the model's use of attention mechanisms provides transparency in decision-making, which is crucial for operational trust and adoption.

There are several avenues for expanding and improving the model. One key direction is to conduct further testing to develop more responsive models, incorporating additional features and more robust datasets. Enhancing our data feed by integrating real-time disaster response programs could make the model more actionable and directly applicable in emergency scenarios. Additionally, future work could involve creating more detailed simulations to compare evacuation speeds across different population groups, considering various evacuation routes and methods. This would enable us to better assess the potential improvements in evacuation efficiency that could be achieved by utilizing our system.

References

[1] D. Y. LIU, D. D. LIU, and Y. ZHOU, "Machine learning for improving air mobility under emergency situations," 2024.

[2] C. C. Branas, E. J. MacKenzie, and C. S. ReVelle, "A trauma resource allocation model for ambulances and hospitals." *Health services research*, vol. 35, no. 2, p. 489, 2000.

[3] S. F. Ghomi and B. Ashjari, "A simulation model for multi-project resource allocation," *International Journal of Project Management*, vol. 20, no. 2, pp. 127–130, 2002.

[4] R. Ma, A. Huang, Z. Jiang, Z. Wang, Q. Luo, and X. Zhang, "A data-driven optimal method for massive passenger flow evacuation at airports under large-scale flight delays," *Reliability Engineering System Safety*, vol. 245, p. 109988, 2024.

[5] S. Baumann, T. Neidhardt, and U. Klingauf, "Evaluation of the aircraft fuel economy using advanced statistics and machine learning," *CEAS Aeronautical Journal*, vol. 12, no. 3, pp. 669–681, 2021.

[6] G. Mishra, S. Mazumdar, and A. Pal, "Improved algorithms for the evacuation route planning problem," in *Combinatorial Optimization and Applications: 9th International Conference, COCOA 2015, Houston, TX, USA, December 18-20, 2015, Proceedings*. Springer, 2015, pp. 3–19.